

Documento de Testes – Backend (Sistema de Saúde)

1. Introdução

Este documento descreve os microcenários de teste aplicados ao backend, incluindo validações de regras de negócio e integridade dos dados, com exemplos práticos em código utilizando pytest e Django.

2. Objetivo

Garantir que:

- As regras de negócio sejam respeitadas
- O sistema lance erros corretamente (ValidationError / IntegrityError)
- O banco de dados mantenha consistência

3. Cenários de Teste com Código

CT01 – Médico da UBS sem vínculo com UBS

Descrição:

Um médico do tipo UBS deve obrigatoriamente estar vinculado a uma UBS.

Código de Teste:

```
def test_medico_ubs_sem_ubs(db):  
    user = User.objects.create(username="medico1")  
    crm = CRM.objects.create(numero="123")
```

```

medico = Medico(
    user=user,
    crm=crm,
    tipo=Medico.Tipo.UBS,
    ubs=None
)

with pytest.raises(ValidationError) as exc:
    medico.save()

assert "ubs" in exc.value.message_dict

```

Resultado Esperado:

Erro de validação informando que o médico deve estar vinculado a uma UBS.

CT02 – UBS sem nome

Descrição:

Uma UBS não pode ser criada sem nome.

Código de Teste:

```

def test_ubs_sem_nome(db):
    ubs = UnidadeBasicaDeSaude(
        nome=None,
        cnes="1234567",
        cidade="Patos"
    )

    with pytest.raises(ValidationError) as exc:
        ubs.full_clean()

    assert "nome" in exc.value.message_dict

```

Resultado Esperado:

Erro de validação indicando que o campo nome é obrigatório.

CT03 – Paciente sem CPF

Descrição:

O paciente deve possuir CPF válido.

Código de Teste:

```
def test_paciente_sem_cpf(db):
    ubs = UnidadeBasicaDeSaude.objects.create(
        nome="UBS",
        cnes="1111111",
        cidade="Patos"
    )

    paciente = Paciente(
        user=User.objects.create(username="p1"),
        ubs=ubs,
        nome="Paciente",
        cpf=None,
        data_nascimento="2000-01-01",
        telefone="999",
        endereco="Rua A",
        sexo="M"
    )

    with pytest.raises(ValidationError) as exc:
        paciente.full_clean()

    assert "cpf" in exc.value.message_dict
```

Resultado Esperado:

Erro indicando que o CPF é obrigatório.

CT04 – Atendimento com paciente de outra UBS

Descrição:

Paciente e médico devem pertencer à mesma UBS do atendimento.

Código de Teste:

```
def test_paciente_ubs_diferente(db):
    ubs1 = UnidadeBasicaDeSaude.objects.create(nome="UBS1",
cnes="1111111", cidade="Patos")
    ubs2 = UnidadeBasicaDeSaude.objects.create(nome="UBS2",
cnes="2222222", cidade="Patos")

    paciente = Paciente.objects.create(
        user=User.objects.create(username="p1"),
        ubs=ubs1,
        nome="Paciente",
        cpf="12345678901",
        data_nascimento="2000-01-01",
        telefone="999",
        endereco="Rua A",
        sexo="M"
    )

    medico = Medico.objects.create(
        user=User.objects.create(username="m1"),
        crm=CRM.objects.create(numero="999"),
        tipo=Medico.Tipo.UBS,
        ubs=ubs1
    )

    atendimento = Atendimento(
        paciente=paciente,
        medico=medico,
        ubs=ubs2,
        observacao_clinica="Teste"
    )

    with pytest.raises(ValidationError):
        atendimento.save()
```

Resultado Esperado:

Erro informando inconsistência de UBS entre paciente/médico e atendimento.

● CT05 – Atendimento sem observação clínica

Descrição:

O atendimento deve possuir observação clínica.

Código de Teste:

```
def test_atendimento_sem_observacao(db):
    # estrutura mínima omitida (já validada antes)

    atendimento = Atendimento(
        paciente=paciente,
        medico=medico,
        ubs=ubs,
        observacao_clinica=None
    )

    with pytest.raises(ValidationError) as exc:
        atendimento.full_clean()

    assert "observacao_clinica" in exc.value.message_dict
```

Resultado Esperado:

Erro indicando campo obrigatório.

● CT06 – Teleconsulta sem atendimento

Descrição:

Toda teleconsulta deve estar vinculada a um atendimento.

Código de Teste:

```
def test_teleconsulta_sem_atendimento(db):
    tele = Teleconsulta(
        atendimento=None
    )

    with pytest.raises(ValidationError) as exc:
        tele.full_clean()

    assert "atendimento" in exc.value.message_dict
```

Resultado Esperado:

Erro indicando campo obrigatório.

CT07 – Solicitação de exame sem origem

Descrição:

A solicitação deve estar vinculada a uma teleconsulta ou retorno.

Código de Teste:

```
def test_solicitacao_sem_origem(db):
    solicitacao = SolicitacaoExame(
        descricao="Exame"
    )

    with pytest.raises(ValidationError) as exc:
        solicitacao.full_clean()

    assert "__all__" in exc.value.message_dict
```

Resultado Esperado:

Erro informando ausência de origem da solicitação.

CT08 – CRM duplicado

Descrição:

Não deve existir dois CRMs iguais.

Código de Teste:

```
from django.db import IntegrityError

def test_crm_duplicado(db):
    CRM.objects.create(numero="12345")

    with pytest.raises(IntegrityError):
        CRM.objects.create(numero="12345")
```

Resultado Esperado:

Erro de integridade do banco (IntegrityError).

4. Critérios de Aceitação

- Todos os testes devem **falhar corretamente quando regras forem violadas**
- Nenhuma inconsistência deve ser permitida no banco
- Erros devem ser **claros, específicos e previsíveis**
- Validações devem ocorrer **antes da persistência no banco**

5. Conclusão

Os testes implementados garantem a confiabilidade do sistema, validando regras críticas e protegendo a integridade dos dados, especialmente em cenários de ausência de informações obrigatórias.